

Active learning

Unlabelled inputs are cheap and labels are not. Let the posterior's uncertainty choose which input to label next.

THE QUESTION

With a large pool of unlabelled inputs and money for a few labels, which input should we label next?

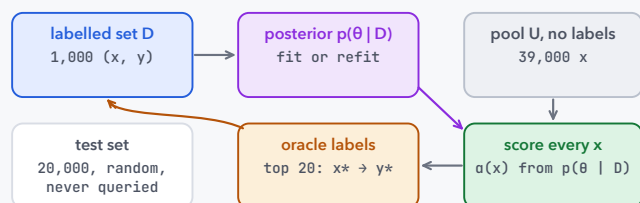
THE INTUITION

A Bayesian model knows where it is unsure. Label where plausible parameter values disagree most: that label removes the most uncertainty about θ .

KEY TAKEAWAY

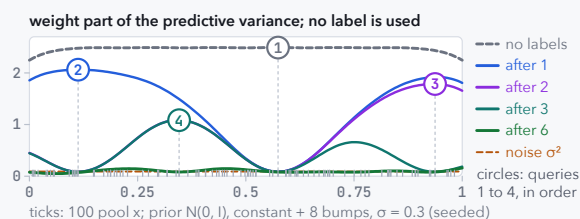
Label $x^* = \arg \max \alpha(x)$, refit, repeat. Regression: largest $\varphi_*^T S_N \varphi_*$. Classification: BALD = $H[\mathbb{E}_\theta p] - \mathbb{E}_\theta H[p]$.

1 Pool-based loop: fit, score the pool, label, refit



The slides' one idea: an acquisition function $\alpha(x)$ says how useful the label of x would be. Score the pool, send $x^* = \arg \max \alpha(x)$ to the oracle, add (x^*, y^*) to $\mathcal{D}$, refit. CIFAR-10 notebook (quoted): 1,000 labels give 35.3% test accuracy, all 40,000 give 61.1%; the loop adds 20 a round for 50 rounds.

2 Regression: label where the weight variance is largest

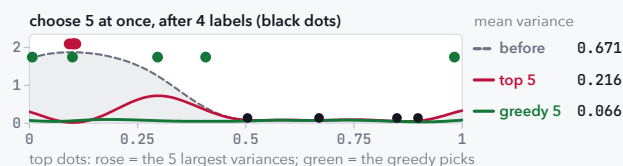


The predictive variance is $\sigma^2 + \varphi_*^T S_N \varphi_*$ (see the Bayesian linear regression sheet); σ^2 is constant, so rank by the weight part. $S_N = (S_0^{-1} + \sigma^{-2} \Phi^T \Phi)^{-1}$ uses inputs only: the whole query plan exists before any label arrives. Labelling φ_* is a rank-one update,

$$S_N \leftarrow S_N - \frac{S_N \varphi_* \varphi_*^T S_N}{\sigma^2 + \varphi_*^T S_N \varphi_*}$$

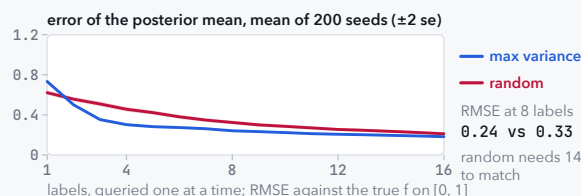
so there the variance falls from v to $v\sigma^2/(\sigma^2 + v)$ (2.49 to 0.087). Queries fill the gaps.

3 A batch: pick, update without y , pick again



The 5 largest variances sit side by side ($x = 0.092$ to 0.107) and teach nearly the same thing. Greedy picks apply the rank-one update in between, need no y , and equal asking one at a time. Classifiers need BatchBALD or a diversity term, such as the notebook's feature-space distance to the nearest label.

4 Largest variance is most information, and it beats random here



For a Gaussian model a label at φ_* carries $I(y_*; \theta | \mathcal{D}) = \frac{1}{2} \log_2(1 + \varphi_*^T S_N \varphi_* / \sigma^2)$ bits about the weights (2.42 for the first query): this is BALD for regression (panel 7). Over 200 seeds, 8 queried labels give RMSE 0.24; random needs 14. The plan trusts the model: refit σ or the features from labels and it is no longer free.

5 Three scores for an unsure classifier

$p(y | x, \mathcal{D})$, 3 classes (illustrative)

		A	B	C	D
least confident	largest	0.10	0.51	0.60	0.40
margin	smallest	0.85	0.00	0.10	0.22
entropy, bits	largest	0.57	1.12	1.57	1.09

green: the point each score labels; rose: what a descending margin sort picks

Least confident $1 - \max_k p_k$; margin $p_{(1)} - p_{(2)}$, where the smallest wins; entropy $-\sum_k p_k \log_2 p_k$, at most $\log_2 3 = 1.585$ bits. Margin sees only the top two, so it prefers B; the others prefer C. With 2 classes all three rank alike. None can tell label noise from an unsure model (page 2). The notebook's margin sort is descending: it labels the most confident (A).

6 The scores in NumPy

```

def H(p):
    # bits, 0 log 0 = 0
    q = np.where(p > 0, p, 1.0)
    return -(p * np.log2(q)).sum(-1)

def scores(P):
    # P: (T, n, K) probs
    p = P.mean(0) # p(y | x, D)
    s = np.sort(p, -1) # ascending
    return dict(
        least_conf=1 - s[:, -1],
        neg_margin=s[:, -2] - s[:, -1],
        entropy=H(p),
        bald=H(p) - H(P).mean(0))

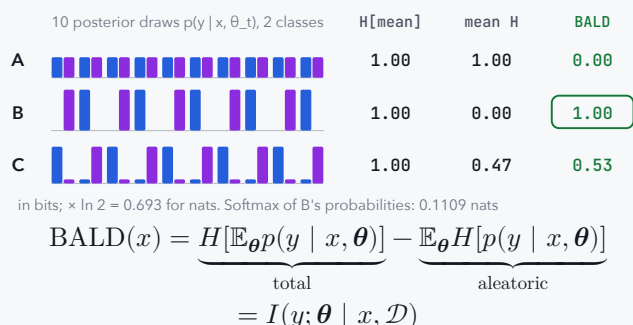
v = (Phi @ SN * Phi).sum(1) # regression
  
```

Label the argmax of any score (hence the negated margin). Pass probabilities: softmax is for logits.

Label where plausible models disagree

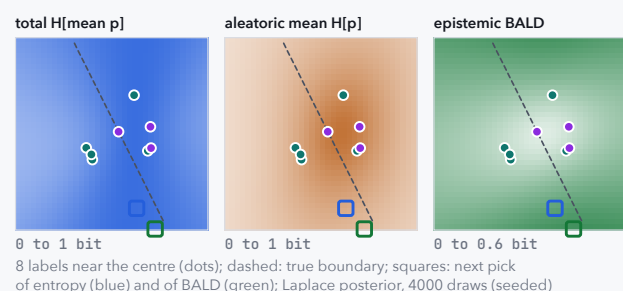
Total uncertainty is noise the label cannot remove plus disagreement between parameter values that the label settles. Pay only for the second.

7 BALD: the information one label gives about θ



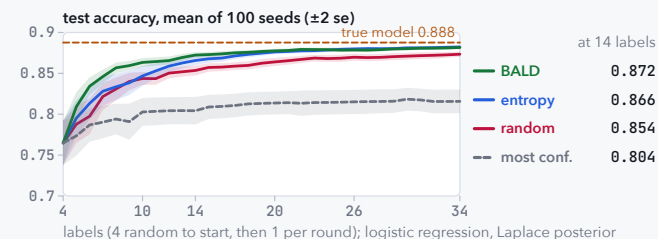
The mutual information of label and parameters (see the information theory sheet) is the epistemic part. All three models predict (0.5, 0.5) with entropy 1 bit, so entropy sampling ties them. In A each draw says 50/50: the label is noise, BALD 0. In B the draws disagree: 1 bit. The notebook's BALD_score applies softmax to the toy's probabilities, (0, 1) $\rightarrow$ (0.27, 0.73), and prints 0.1109 nats for B; its pool sampler passes logits and is right.

8 Epistemic uncertainty grows where the data stop



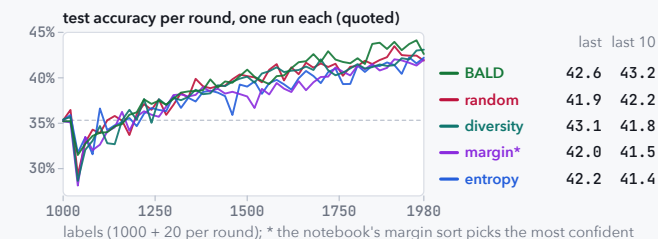
Bayesian logistic regression with a Laplace posterior (see the Laplace approximation sheet), 8 labels near the centre. Total entropy is high along the whole boundary. The aleatoric part (up to 0.90 bit) sits near the data, where every draw says about 50/50; the epistemic part (up to 0.54 bit) grows along the boundary away from the data, where the draws tilt apart. BALD's next pick scores 0.53, entropy's 0.44. Draws of θ can come from Laplace, MCMC, an ensemble or MC dropout (dropout left on at prediction, as in the MC dropout notebook).

9 On a 2D problem the gain is real but small



Truth $p(y = 1 | x) = \sigma(-0.5 + 3x_1 + 1.5x_2)$ on $[-2, 2]^2$; pool 500, test 2,000, 100 seeds. At 14 labels entropy leads random by 1.2 ± 0.4 points (paired); random needs 22 labels to match it. BALD leads entropy early (1.7 ± 0.6 points at 10) and ties from 20. Labelling the most confident points, as the reversed margin sort does, loses to random. By 34 labels the other three are within 1.5 points of the true model's 0.888.

10 The notebook's CIFAR-10 run cannot rank the strategies



ResNet-18, quoted. Last printed values: 41.9% (random) to 43.1% (diversity). Means of the last 10 rounds: 41.4% (entropy) to 43.2% (BALD), random 42.2%. Single rounds swing by up to 7.2 points, each strategy ran once, and each round fine-tunes the same network for 20 more epochs. Nothing separated from random here; a claim needs several seeds.

11 Pause and think

Does regression need y to pick the next x ? No, if σ and the prior are fixed.

Why not always take the largest entropy? It returns to coin-flip labels (A in panel 7), where BALD is 0.

Can active learning lose to random? Yes: a wrong model, repetitive top- k batches, a reversed sort.

And the first labels? With 4 the posterior is poor (cold start): pick them at random.

$\alpha(x)$ ranks the pool · regression: largest $\varphi_*^T S_N \varphi_*$, no labels needed · BALD = $H[\mathbb{E}p] - \mathbb{E}H[p] = I(y; \theta)$, the epistemic part · compare with random over several seeds

Source: lecture slides "Active learning" (one frame: the acquisition function) and the active learning and MC dropout notebooks, nipunbatra.github.io/pml-teaching. CIFAR-10 numbers are quoted; the rest is NumPy with fixed seeds and goes beyond the slides. Notebook corrections: the margin sampler sorts descending, so it labels the most confident points; BALD_score applies softmax to the toy's probabilities (B prints 0.1109 nats; it is $\ln 2 = 0.693$ nats = 1 bit); aL_loop prints the accuracy from before each round's retraining, so its 50 values cover 1,000 to 1,980 labels; random sampling reseeds on every call; the runs start at 35.21% to 35.42% though each copies one model.